The Telephone Contact Management System

Source Code:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for a Contact
typedef struct Contact {
    char name[50];
    char phone[15];
    struct Contact *next;
    struct Contact *prev;
} Contact;

Contact *head = NULL;
Contact *current = NULL; // To track "back and forth" movement

// Function Prototypes
void addContact();
void deleteContact();
void searchContact();
void updateContact();
void displayCurrent();
void moveNext();
void movePrev();

int main() {
```

```
    int choice;
    while (1) {
        printf("\n--- TELEPHONE CONTACT LIST ---\n");
        displayCurrent();
        printf("1. Add Contact\n2. Delete Contact\n3. Search\n4. Update\n5. Next
Contact\n6. Previous Contact\n7. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar(); // Clear buffer

        switch (choice) {
            case 1: addContact(); break;
            case 2: deleteContact(); break;
            case 3: searchContact(); break;
            case 4: updateContact(); break;
            case 5: moveNext(); break;
            case 6: movePrev(); break;
            case 7: exit(0);
            default: printf("Invalid choice!\n");
        }
    }
    return 0;
}

void addContact() {
    Contact *newNode = (Contact *)malloc(sizeof(Contact));
    printf("Enter Name: ");
    fgets(newNode->name, 50, stdin);
    newNode->name[strcspn(newNode->name, "\n")] = 0; // Remove newline
```

```
    printf("Enter Phone: ");
    fgets(newNode->phone, 15, stdin);
    newNode->phone[strcspn(newNode->phone, "\n")] = 0;

    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        current = head;
    } else {
        Contact *temp = head;
        while (temp->next != NULL) temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("Contact added successfully!\n");
}

void displayCurrent() {
    if (current == NULL) {
        printf("[ List is Empty ]\n");
    } else {
        printf("[ Current Contact: %s | %s ]\n", current->name, current->phone);
    }
}

void moveNext() {
    if (current != NULL && current->next != NULL) {
```

```
        current = current->next;
    } else {
        printf("End of list.\n");
    }
}

void movePrev() {
    if (current != NULL && current->prev != NULL) {
        current = current->prev;
    } else {
        printf("Start of list.\n");
    }
}

void searchContact() {
    char name[50];
    printf("Enter name to search: ");
    fgets(name, 50, stdin);
    name[strcspn(name, "\n")] = 0;

    Contact *temp = head;
    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            printf("Found: %s - %s\n", temp->name, temp->phone);
            current = temp; // Move pointer to searched contact
            return;
        }
        temp = temp->next;
    }
```

```
    printf("Contact not found.\n");
}

void deleteContact() {
    if (current == NULL) return;

    Contact *toDelete = current;

    if (toDelete->prev != NULL) toDelete->prev->next = toDelete->next;
    if (toDelete->next != NULL) toDelete->next->prev = toDelete->prev;
    if (toDelete == head) head = toDelete->next;

    // Move current pointer
    if (toDelete->next != NULL) current = toDelete->next;
    else current = toDelete->prev;

    free(toDelete);
    printf("Contact deleted.\n");
}

void updateContact() {
    if (current == NULL) return;
    printf("Updating current contact (%s)...\n", current->name);
    printf("Enter New Phone: ");
    fgets(current->phone, 15, stdin);
    current->phone[strcspn(current->phone, "\n")] = 0;
    printf("Updated!\n");
}
```